

PaliGemma · arXiv 2407.07726 · Sep 16

Visionary

seat 5 · 11+3

2026: adaptation > backbone swap

Paper club · VLA Architectures · dark academic

Visionary agenda · Sep 16

1

Adaptation > backbone swap

2

Multi-cam 224 + selective 448 foveation

3

PG = visual tokenizer + residual LLM

4

Action expert / noise / stages / data = upside

5

Stay until better open ego/video 3-4B appears

2026 adaptation map

Upside = adaptation — not backbone replacement

Action expert

Lightweight policy tower
on frozen / LoRA PG
prefix — π_0 pattern

Noise / stages

Diffusion / flow matching
on actions; System-2
stages & recovery

Data

Egocentric / multi-cam
BEHAVIOR rollouts;
task & stage embeddings

Multi-cam 224

Default latency budget;
K-frame concat before
blanket 896

Selective 448

Foveation for OCR /
fine grasp / HUD read
— not everywhere

Visual tokenizer

PG as eyes + residual
LLM; free language →
task/stage embeddings

Multi-cam 224 + selective 448 foveation

> blanket 896 everywhere

Default: multi-cam @224

K cameras × 256 tokens
keeps LLM context sane
Latency-friendly for control
History via frame concat

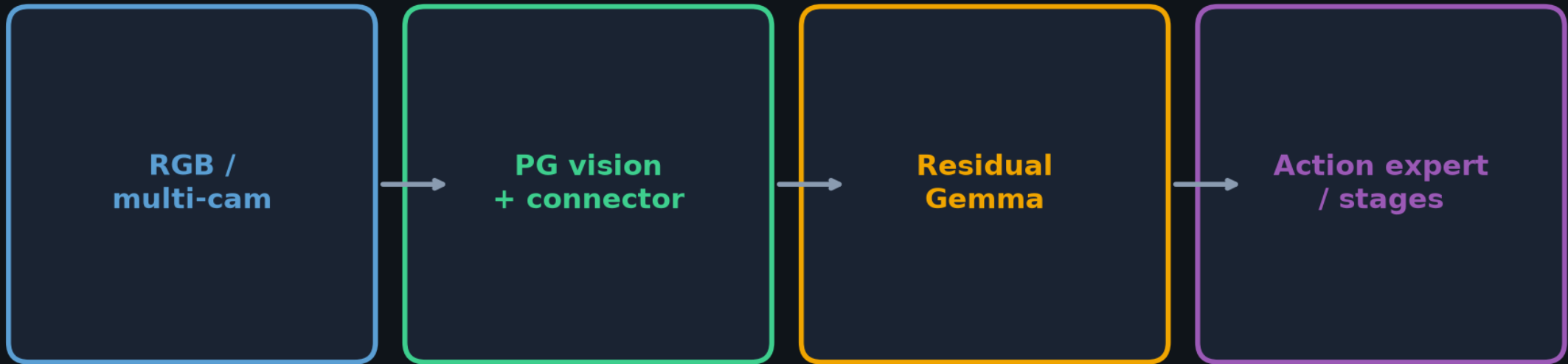
Escalate: selective 448

OCR panels / HUDs
Fine grasp affordances
Widget / UI grounding
Not every frame

Anti-pattern

Blanket 896 on all cams = token explosion + latency hit.
Winners already trade free language for task/stage embeddings — keep PG as visual tokenizer + residual LLM.

PaliGemma as visual tokenizer + residual LLM

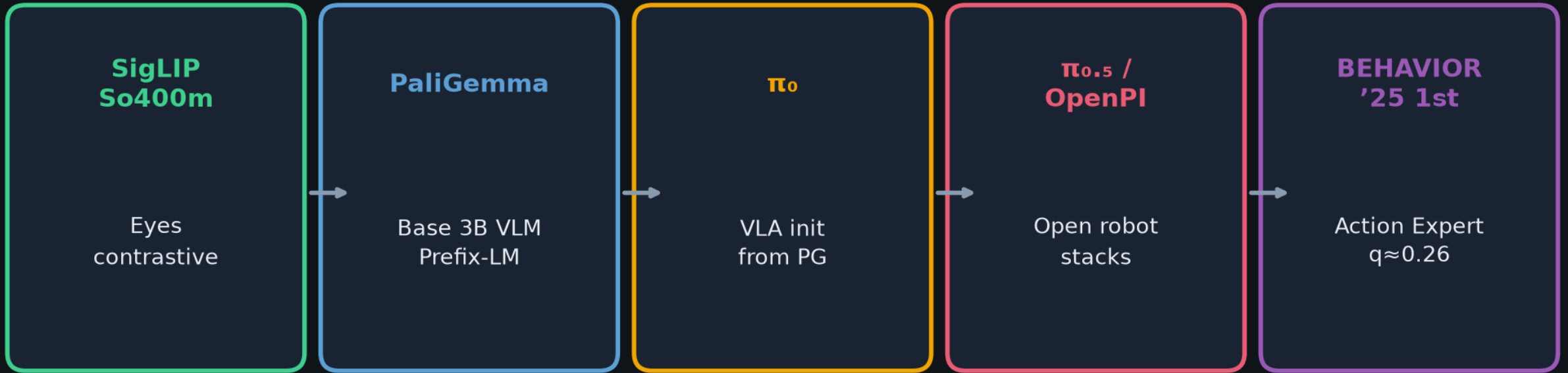


2026 thesis

PaliGemma-class remains default open VLA init until a better open egocentric/video 3-4B appears. Differentiate on adaptation, not backbone swap.

Club chain · SigLIP → PaliGemma → Action Expert

Why an open 3B base VLM is load-bearing for this club's robot FM work



PaliGemma = the critical open bridge

Closed VLMs cannot seed OpenPI forks. Loc/seg + OCR@448+ = grounding API.
Failures often sit in action expert / stages / data — but these are the eyes.

Club thread · perception vs planner

PaliGemma IS

- ▶ Open base VLM for transfer
- ▶ Loc/seg + OCR grounding API
- ▶ Default π_0 / OpenPI eyes+language
- ▶ Prefix-LM observe+goal shaping

PaliGemma is NOT

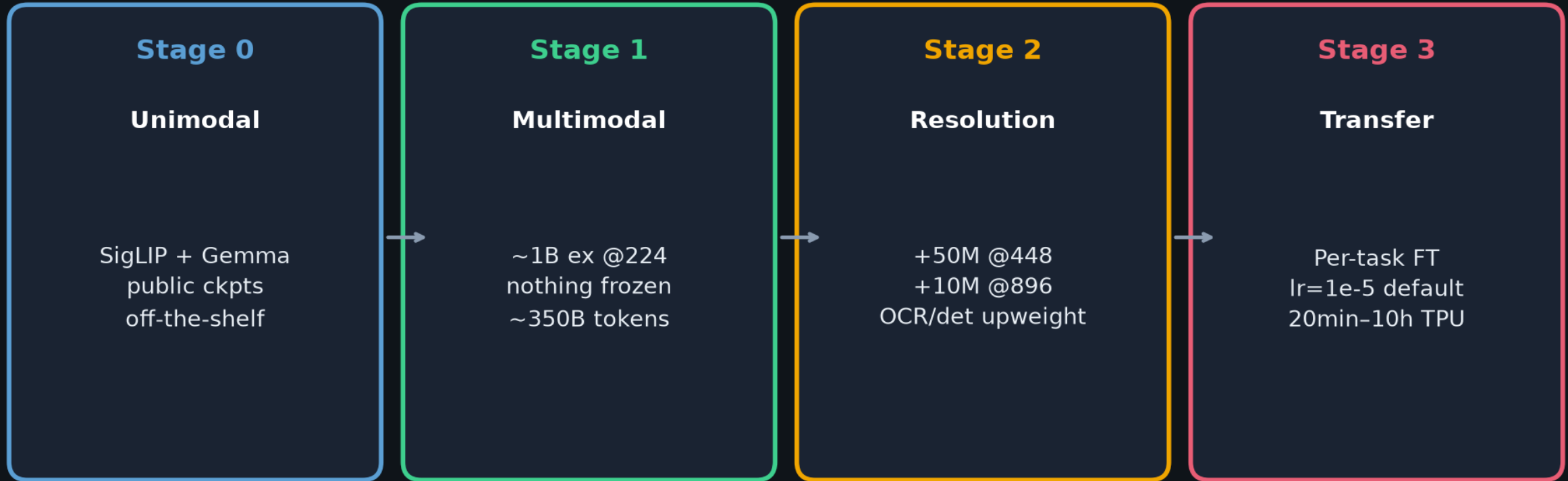
- ▶ A classical motion planner
- ▶ A closed-loop policy
- ▶ An instruction-tuned chat bot
- ▶ Proof of BEHAVIOR wins alone

Hand-offs

Empiricist: LoRA / freeze / res ladder pressure-tests.
Visionary: adaptation (action expert, stages, data) > backbone swap in 2026.

Training stages 0 → 3

Unimodal priors → multimodal skills → resolution → per-task transfer



Key Stage1 choice: do NOT freeze ViT (slow LR warmup) — freezing LM hurts a lot

Prefer released paligemma-pt-{224,448} as substrate for π_0 / OpenPI / BEHAVIOR forks

the pretraining datasets [55].

Largely following previous PaLI works, these are the pretraining tasks:

`caption {lang}`

We include the simple captioning objective on various datasets, including WebLI in over 100 languages, and CC3M-35L. Previous PaLIs use an encoder-decoder language model with the Split-Cap objective, however for PaliGemma with the decoder-only language model, plain captioning is a more informative and simpler objective.

`ocr`

Concatenation (in raster order) of all text on the image transcribed by a public OCR system. Potentially skipping random snippets of OCR in order to fit sequence length without biasing recognition towards the beginning of raster order.

`answer en {question}`

Generated VQA on CC3M-35L following [19] with questions in 35 languages but English answers. Additionally, English-only object-centric questions on OpenImages following [91]:

listing: What objects are in the image?,
presence: Is {thing} in the image?,
multi-object presence: Which of {thing},
{thing}... are in the image?,
and newly, counting: How many {thing}?

`question {lang} {English answer}`

Generated VQG on CC3M-35L following [19] generating questions in 35 languages, for a given English answer.

`detect {thing} ; {thing} ; ...`

Multi-object detection similar to Pix2Seq [22] on generated open-world data via pseudo-labeling as described in OWL-ViT2 [83].

`segment {thing} ; {thing} ; ...`

Multi-object instance segmentation as in PaLI-3 [25] on generated open-world data similar to OWL-ViT2 [83] and SAM [54].

`caption <ymin><xmin><ymax><xmax>`

Grounded captioning of what is in the box, following LocCa [115]. The box is indicated by the same location tokens as used in detection and segmentation: normalized image coordinates binned to 1024 tokens.

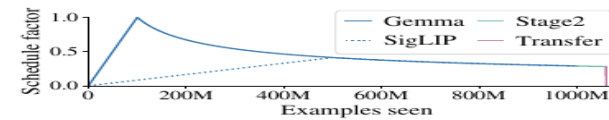


Figure 3 | Learning-rate schedule across stages.

Notably distinct from the widely used LLaVa’s GPT-4 generated instruction following data, none of PaliGemma’s pretraining tasks is the output of a larger commercial VLM.

Finally, we believe that it is important to detect and remove all images in our pretraining datasets which are near-duplicates of images in the transfer tasks we evaluate in this report [55], as well as a few more popular computer vision benchmarks. Doing so, we more accurately capture PaliGemma’s capability to transfer to new tasks.

3.2.6. Other pretraining details

Throughout pretraining, we use an “infinite” learning-rate schedule following [131], which provides a straightforward way of chaining several stages without decaying the learning-rate between them. Figure 3 shows the full schedule: pretraining is one continuous rsqrt curve for all stages. The transfer can then act as a cooldown, fully annealing the learning rate. We recommend transferring with a simple setup that tunes the full model using a cosine learning-rate schedule with a short linear warm-up and decaying to zero. This is not well represented by Figure 3 due to its comparatively short duration.

The model was entirely trained in the open-source `big_vision` codebase [12] on Cloud TPUv5e [38]. However, some of the pretraining datasets remain private. During training, we partition data, as well as model parameters and optimizer state (Zero-DP style [96]) across all available devices using JAX [16] with GSPMD [125]. This fully-sharded data-parallel (FSDP [137]) sharding strategy is achieved by constructing global arrays and annotating the sharding accordingly, with the XLA compiler [97] taking care of the concrete implementation of the computation

Visionary close

Adaptation > backbone swap · 224 + selective 448 · tokenizer view